Frontend Design

AI-Powered Security Analytics Platform

Field	Value
Document ID	FRONTEND-001
Version	1.0
Date	2026-07-18
Status	Draft
Framework	Next.js 14+ (App Router)
Language	TypeScript
Architecture Reference	ADR-001
Backend API	BACKEND-002

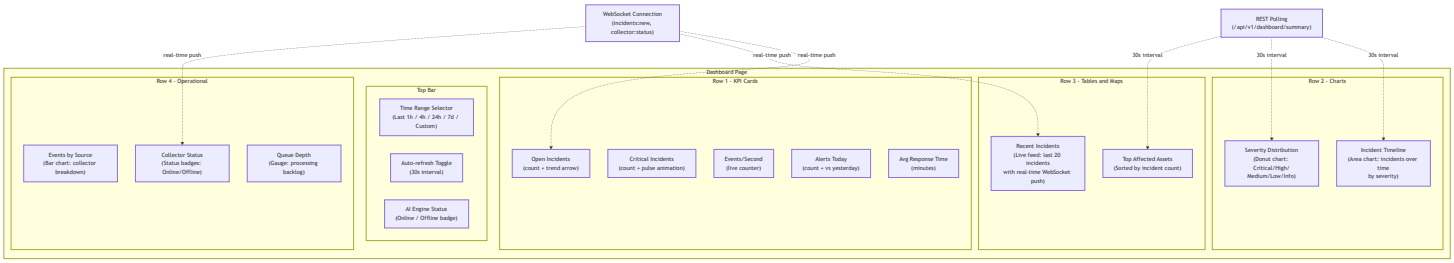
Table of Contents

1. Dashboard
2. Pages
3. Components
4. Charts
5. Authentication
6. Role-Based UI
7. Folder Structure

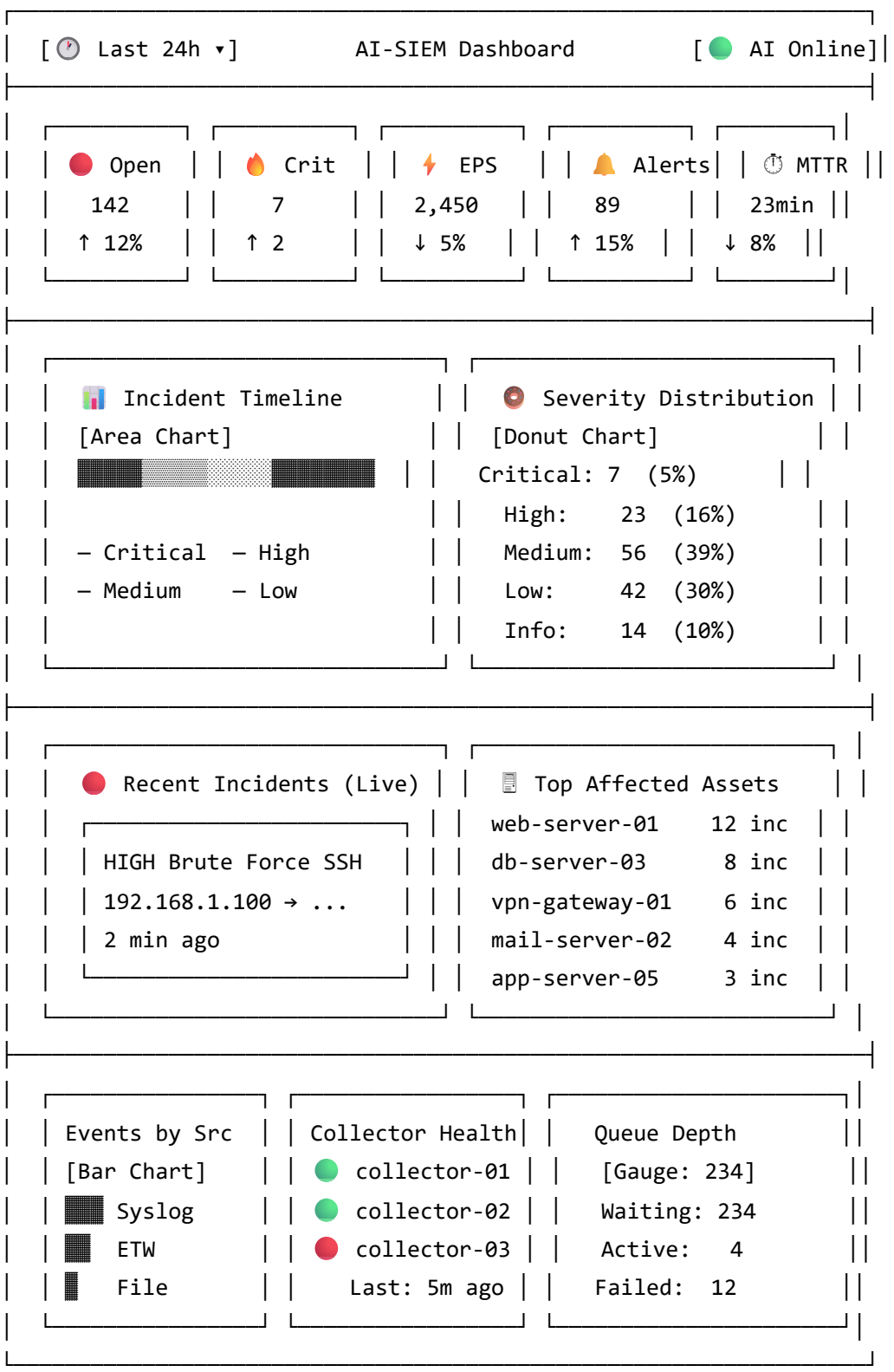
1. Dashboard

1.1 Dashboard Architecture

The Dashboard (`/`) is the primary landing page for SOC Analysts. It provides a **real-time operational overview** of the security posture through live-updating widgets powered by WebSocket subscriptions and periodic REST polling.



1.2 Dashboard Wireframe Layout

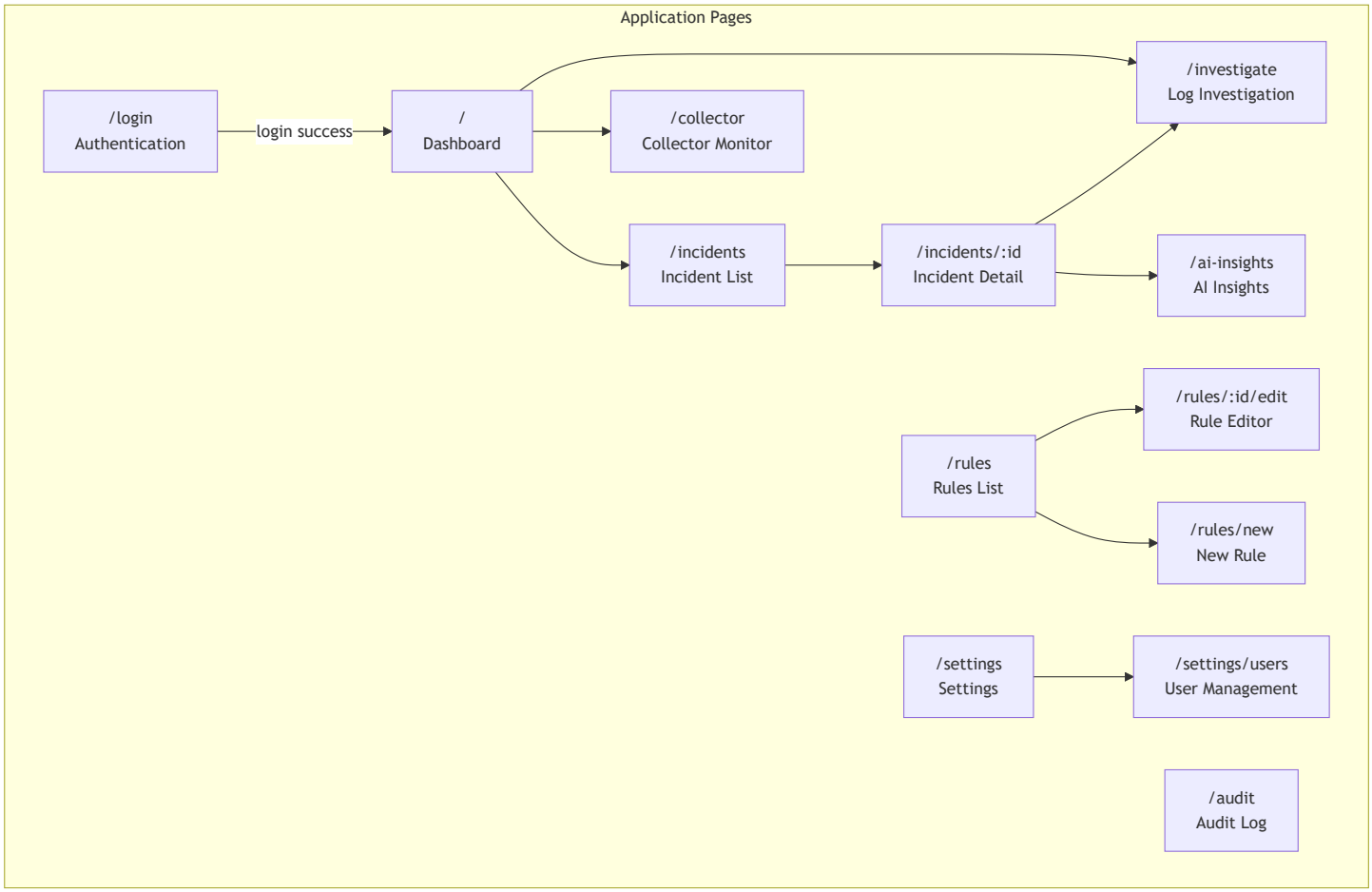


1.3 Dashboard Data Sources

Widget	Data Source	Update Method	Endpoint
Open Incidents KPI	REST + WebSocket	WebSocket push on `incidents:new` / `incidents:status`	`GET /api/v1/dashboard/summary`
Critical Incidents KPI	REST + WebSocket	WebSocket push	`GET /api/v1/dashboard/summary`
Events/Second	REST	Poll every 10s	`GET /api/v1/dashboard/summary`
Alerts Today	REST	Poll every 30s	`GET /api/v1/dashboard/summary`
Avg Response Time	REST	Poll every 60s	`GET /api/v1/dashboard/summary`
Incident Timeline	REST	Poll every 30s	`GET /api/v1/dashboard/timeline?range=24h`
Severity Distribution	REST	Poll every 30s	`GET /api/v1/dashboard/summary`
Recent Incidents	WebSocket	Real-time push	`WS /ws` (channel: `incidents:new`)
Top Affected Assets	REST	Poll every 60s	`GET /api/v1/dashboard/top-assets`
Events by Source	REST	Poll every 60s	`GET /api/v1/dashboard/source-breakdown`
Collector Health	WebSocket	Real-time push	`WS /ws` (channel: `collector:status`)
Queue Depth	REST	Poll every 15s	`GET /api/v1/collector/status`
AI Engine Status	REST	Poll every 30s	`GET /api/v1/collector/status`

2. Pages

2.1 Page Map



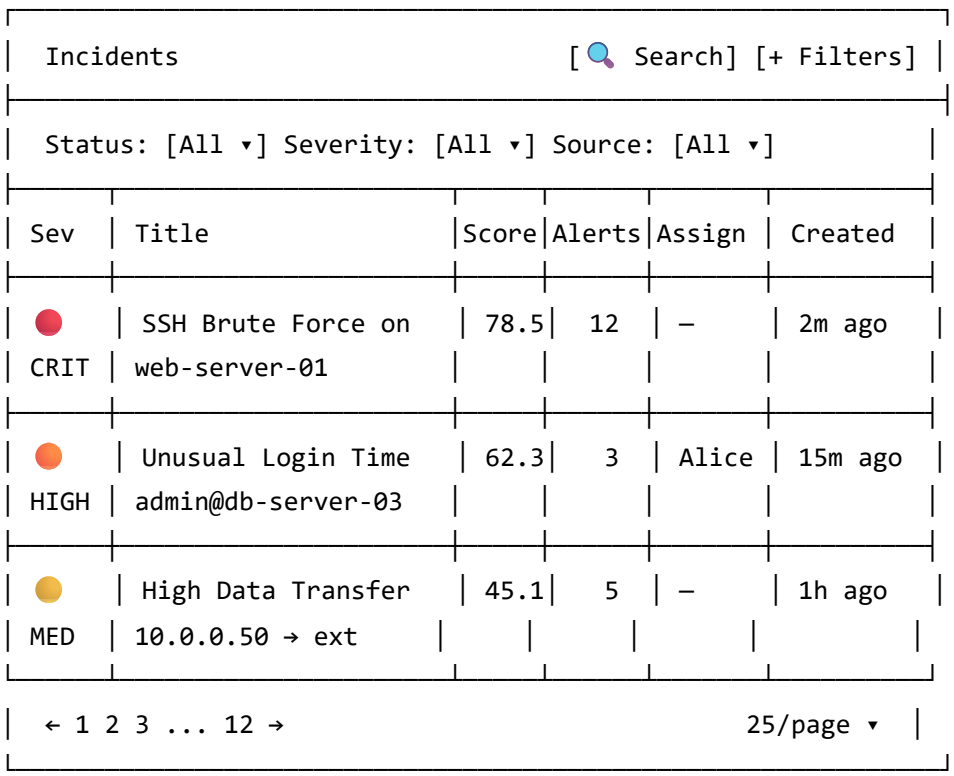
2.2 Page Specifications

`/` — Dashboard

Property	Value
Purpose	Real-time SOC operational overview
Access	All authenticated roles
Data	REST polling + WebSocket real-time
Key interactions	Click incident → navigate to `/incidents/:id`. Click asset → navigate to `/investigate?entity=<hostname>`
Refresh	Auto-refresh every 30s. Manual refresh button. WebSocket push for live data

`/incidents` — Incident List

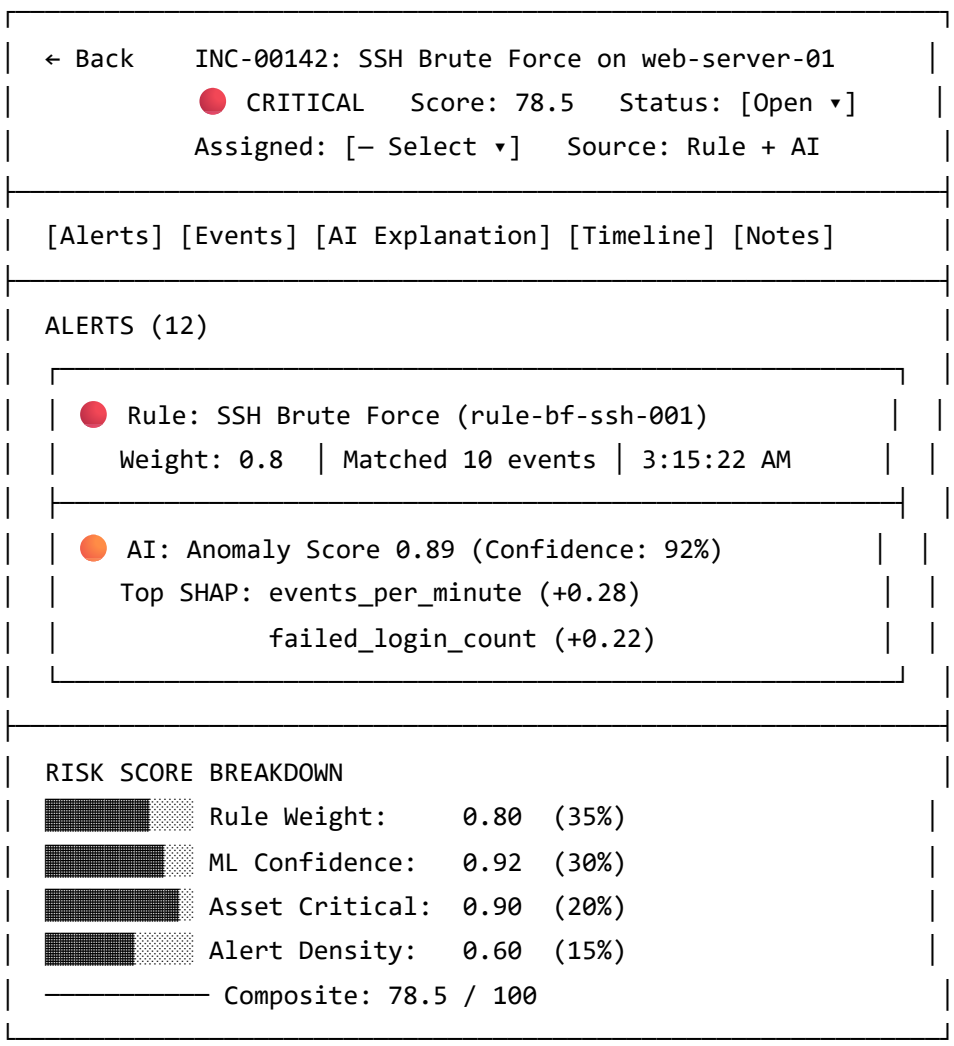
Property	Value
Purpose	Browse, filter, and triage incidents
Access	All authenticated roles
Filters	Status (Open/Investigating/Resolved/Closed), Severity, Source (Rule/AI/Both), Time range, Assigned to, Search text
Sorting	Risk score (desc), Created at (desc), Severity, Alert count
Pagination	25 per page, cursor-based pagination
Bulk actions	Assign to analyst, Change status (Admin/Engineer only)



`/incidents/:id` — Incident Detail

Property	Value
Purpose	Deep-dive into a single incident: alerts, events, AI explanation, timeline, notes

Property	Value
Sections	Header (title, severity, status, assignee), Alert list, Event timeline, AI Explanation panel (SHAP), Risk score breakdown, Notes & activity log
Actions	Change status, Assign analyst, Add note, Pivot to investigation



`/investigate` — Log Investigation

Property	Value
Purpose	Search and explore raw normalized events
Search	Time range, keyword (full-text), source IP, destination IP, username, hostname, severity, OCSF class
Results	Sortable event table with expandable rows showing full OCSF event

Property	Value
Pivoting	Click IP → filter by IP. Click user → filter by user. Click event → view linked incidents
Export	CSV export of search results

`/rules` — Rules List & Editor

Property	Value
Purpose	Manage Sigma-compatible detection rules
List view	Name, type, severity, status (active/disabled), alerts generated, last triggered
Editor	Monaco YAML editor with syntax highlighting, validation errors inline, live preview
Actions	Create, Edit, Enable/Disable, Delete, Test (dry-run), Import, Export
Access	Read: all. Write: Admin + Security Engineer

`/ai-insights` — AI Insights

Property	Value
Purpose	Visualize AI detection results and SHAP explanations
Sections	Anomaly score distribution (histogram), top anomalous events, SHAP waterfall charts, feature importance ranking, model info (version, threshold, training date)
Access	All authenticated roles

`/collector` — Collector Monitor

Property	Value
Purpose	Monitor collector health, queue status, processing metrics
Sections	Collector status cards (Online/Offline/Degraded), files processed counter, events collected counter, queue depth gauge, failed/quarantined files, AI Engine health
Updates	WebSocket real-time for status changes

`/settings` — Settings

Property	Value
Purpose	Application configuration
Sections	Profile settings, notification preferences, data retention settings, AI threshold adjustment
Access	Profile: all users. System settings: Admin only

`/settings/users` — User Management

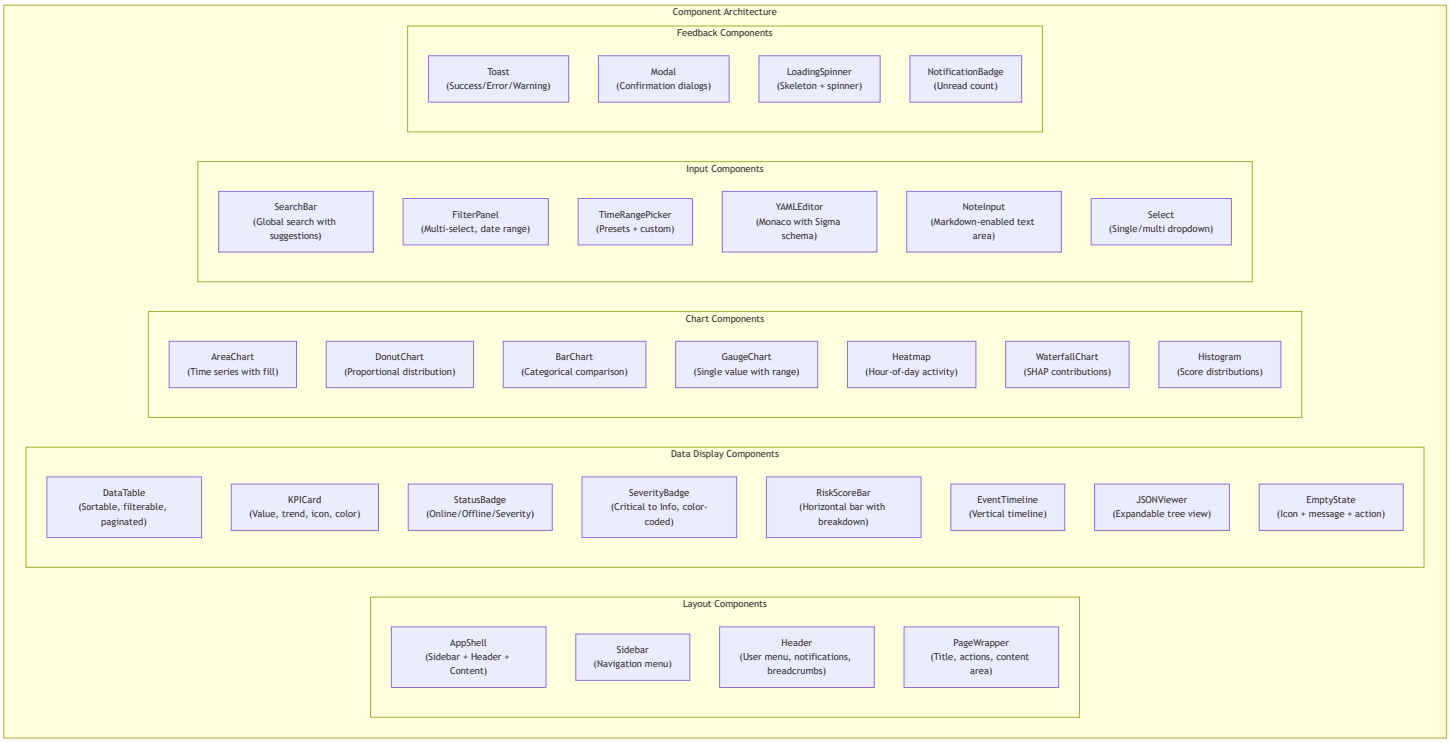
Property	Value
Purpose	Create, edit, deactivate users and assign roles
Access	Admin only
Actions	Create user, Edit user, Change role, Deactivate, Reset password

`/audit` — Audit Log

Property	Value
Purpose	View immutable audit trail of all actions
Filters	Action type, Actor, Target, Time range
Access	Admin + Security Engineer (read-only)

3. Components

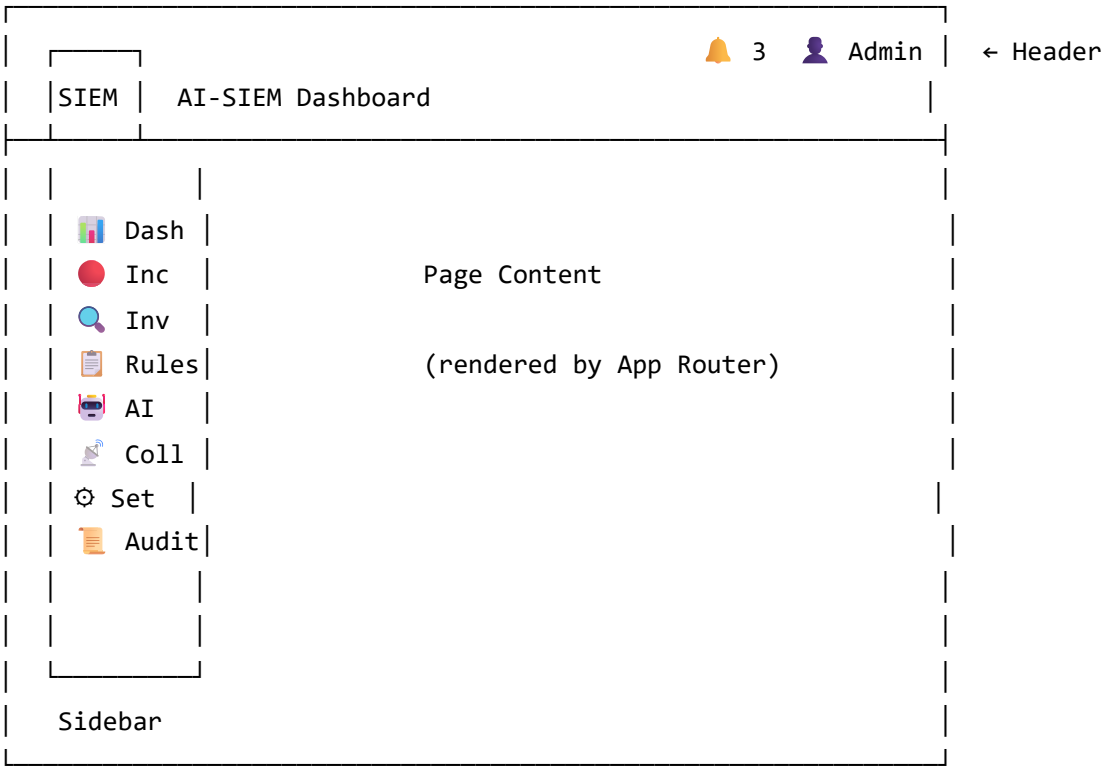
3.1 Component Architecture



3.2 Core Component Specifications

AppShell

The root layout wrapping all authenticated pages.



Property	Detail
Sidebar	Collapsible. Icons + labels. Active page highlighted. Role-filtered items
Header	Breadcrumbs, notification bell (unread count from WebSocket), user avatar + dropdown (profile, logout)
Content	Full-width below header, scrollable. Rendered by Next.js App Router
Responsive	Sidebar collapses to icon-only on medium screens. Hamburger menu on mobile

DataTable

Reusable table component used on Incidents, Rules, Audit, Users, and Investigation pages.

Feature	Implementation
Sorting	Click column header. Ascending/descending toggle. Sort indicator arrow
Filtering	Column-level filters via FilterPanel. Server-side for large datasets
Pagination	Cursor-based. Page size selector (10/25/50/100). Total count display
Row expansion	Expandable rows for inline detail preview

Feature	Implementation
Selection	Checkbox column for bulk actions (where applicable)
Loading	Skeleton rows while loading. Empty state component when no results
Responsive	Horizontal scroll on mobile. Priority columns remain visible

KPICard

Dashboard metric card with trend indicator.

Prop	Type	Description
`title`	`string`	Metric name (e.g., "Open Incidents")
`value`	`number | string`	Current value
`trend`	`number`	Percentage change from previous period
`trendDirection`	`"up" | "down" | "flat"`	Arrow direction
`trendColor`	`"positive" | "negative" | "neutral"`	Green for improvements, red for degradation
`icon`	`ReactNode`	Metric icon
`color`	`string`	Card accent color
`loading`	`boolean`	Show skeleton while data loads

StatusBadge

Status	Color	Animation
Online	Green	Subtle pulse
Degraded	Amber	Slow pulse
Offline	Red	None (static)
Critical	Red	Fast pulse
High	Orange	None

Status	Color	Animation
Medium	Yellow	None
Low	Blue	None
Informational	Grey	None

YAMLEditor

Built on Monaco Editor with Sigma-specific features.

Feature	Implementation
Syntax highlighting	YAML language mode
Auto-completion	Custom Sigma schema suggestions (field names, operators, OCSF classes)
Inline validation	Real-time YAML parse errors highlighted. Sigma schema violations shown as warnings
Line numbers	Always visible
Minimap	Disabled for compact view
Theme	Dark theme matching application design

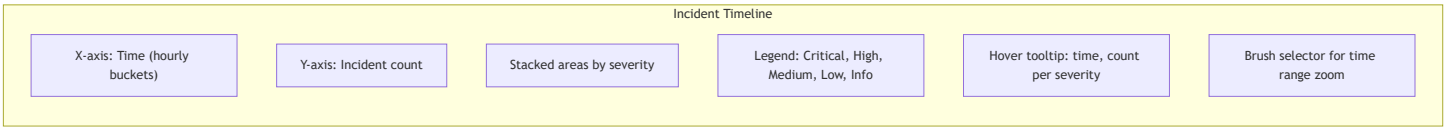
4. Charts

4.1 Chart Library

Library	Version	Purpose
Recharts	2.x	Primary chart library (Area, Bar, Donut, Line, Histogram)
Custom SVG	—	Gauge charts, SHAP waterfall (not natively supported by Recharts)

4.2 Chart Component Specifications

Incident Timeline (Area Chart)



Property	Value
Type	Stacked Area Chart
X-axis	Time (1h/4h/24h/7d depending on selected range)
Y-axis	Incident count
Stacks	Severity levels (5 colors)
Interaction	Hover tooltip, brush zoom, click to filter
Colors	Critical: `#e74c3c` , High: `#e67e22` , Medium: `#f1c40f` , Low: `#3498db` , Info: `#95a5a6`

Severity Distribution (Donut Chart)

Property	Value
Type	Donut Chart
Segments	One per severity level
Center text	Total incident count
Interaction	Hover: segment highlights, shows percentage. Click: filter incidents by severity
Colors	Same severity palette as Timeline

Events by Source (Bar Chart)

Property	Value
Type	Horizontal Bar Chart
X-axis	Event count
Y-axis	Source type (Syslog, Windows ETW, File, HTTP)

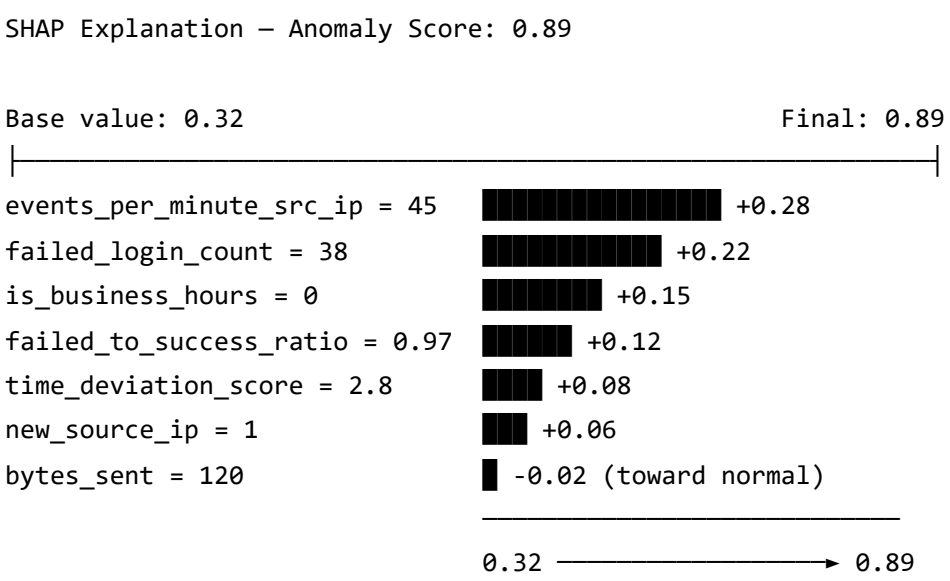
Property	Value
Interaction	Hover tooltip with exact count and percentage

Queue Depth (Gauge Chart)

Property	Value
Type	Radial Gauge (custom SVG)
Range	0 to configurable max (default 1000)
Zones	Green (0-30%), Yellow (30-70%), Red (70-100%)
Center	Current value and label
Animation	Smooth needle transition on value change

SHAP Waterfall (Waterfall Chart)

Property	Value
Type	Horizontal waterfall
Bars	One per SHAP feature, sorted by absolute contribution
Direction	Positive (right, red) pushes toward anomaly. Negative (left, blue) pushes toward normal
Base	Base value (expected model output) shown as starting point
Final	Final anomaly score shown as endpoint
Interaction	Hover: feature name, actual value, SHAP contribution



Anomaly Score Distribution (Histogram)

Property	Value
Type	Histogram
X-axis	Anomaly score (0.0 to 1.0, 20 bins)
Y-axis	Event count
Threshold line	Vertical dashed line at current threshold (0.65)
Highlight	Bins above threshold highlighted in red
Interaction	Hover: bin range, count, percentage

Activity Heatmap

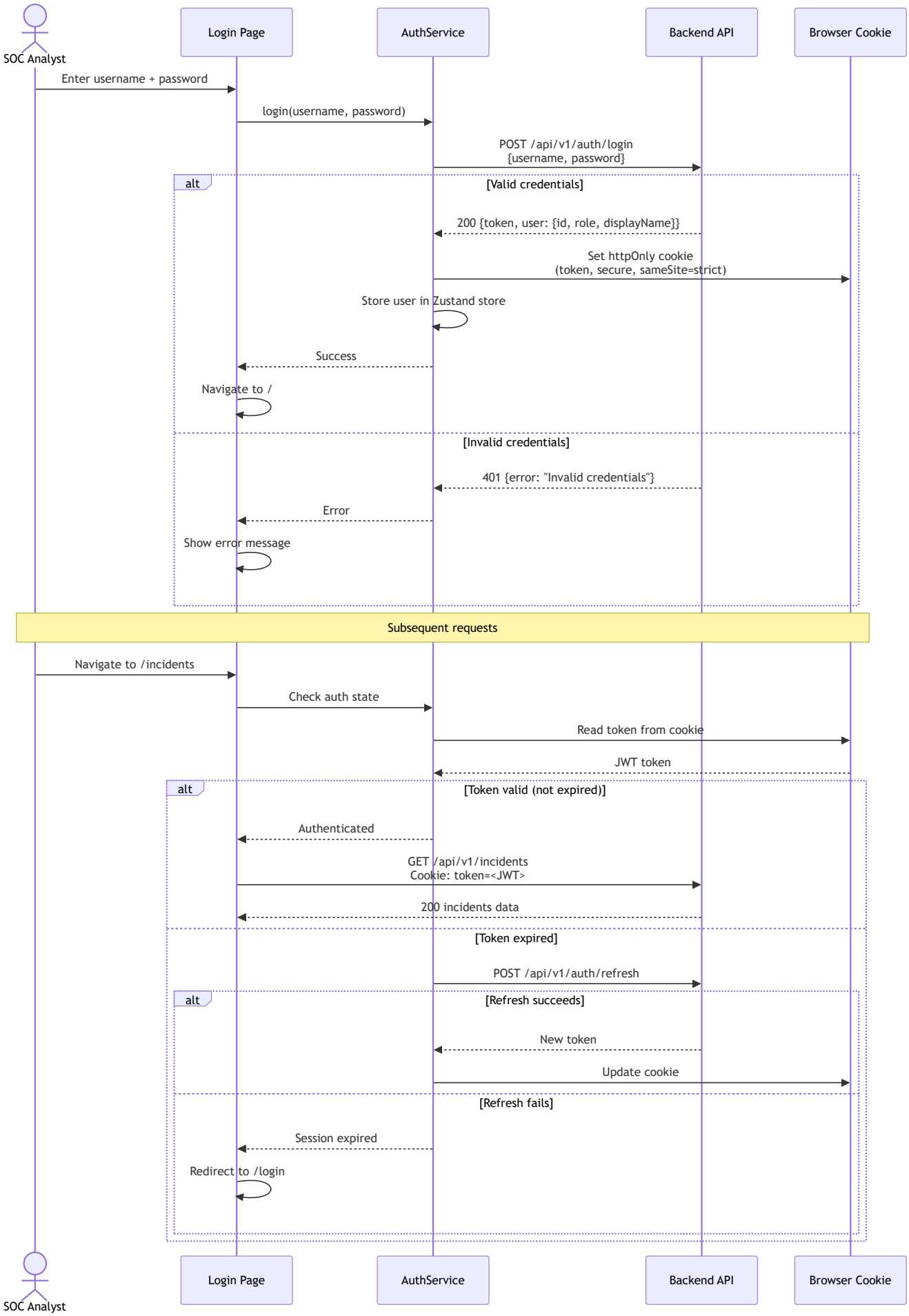
Property	Value
Type	Grid heatmap
X-axis	Hour of day (0-23)
Y-axis	Day of week (Mon-Sun)
Color scale	White (0 events) → Dark red (max events)
Purpose	Visualize when security events concentrate (used in Investigation page)

4.3 Chart Theme

Token	Value	Usage
`chart.critical`	`#e74c3c`	Critical severity
`chart.high`	`#e67e22`	High severity
`chart.medium`	`#f1c40f`	Medium severity
`chart.low`	`#3498db`	Low severity
`chart.info`	`#95a5a6`	Informational
`chart.anomaly`	`#e74c3c`	SHAP positive (toward anomaly)
`chart.normal`	`#3498db`	SHAP negative (toward normal)
`chart.grid`	`#2c3e50`	Chart grid lines
`chart.text`	`#ecf0f1`	Axis labels, legend text
`chart.background`	`#1a1a2e`	Chart container background
`chart.tooltip.bg`	`#16213e`	Tooltip background
`chart.tooltip.text`	`#ffffff`	Tooltip text

5. Authentication

5.1 Authentication Flow


SOC Analyst
Login Page
AuthService
Backend API
Browser Cookie
Enter username + password
login(username, password)
POST /api/v1/auth/login
{username, password}
alt
[Valid credentials]
200 {token, user: {id, role, displayName}}
Set httpOnly cookie
(token, secure, sameSite=strict)
Store user in Zustand store
Success
Navigate to /
[Invalid credentials]
401 {error: "Invalid credentials"}
Error
Show error message
Subsequent requests
Navigate to /incidents
Check auth state
Read token from cookie
JWT token
alt
[Token valid (not expired)]
Authenticated
GET /api/v1/incidents
Cookie: token=<JWT>
200 incidents data
[Token expired]
POST /api/v1/auth/refresh
alt
[Refresh succeeds]
New token
Update cookie
[Refresh fails]
Session expired
Redirect to /login
SOC Analyst
Login Page
AuthService
Backend API
Browser Cookie

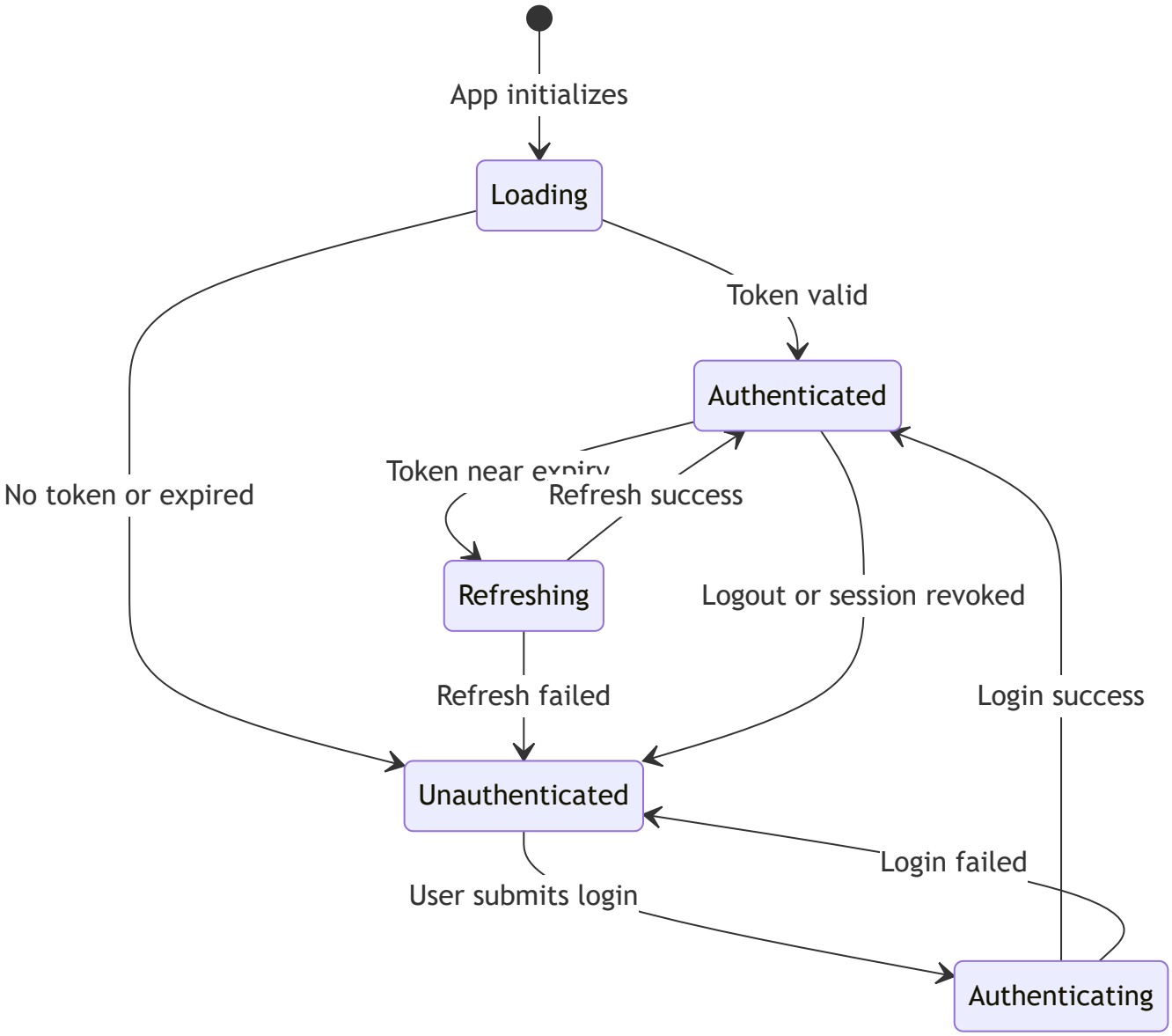
5.2 Authentication Components

Component	Purpose
LoginPage	Login form with username/password. Error display. Loading state
AuthProvider	React Context wrapping the app. Provides `user`, `isAuthenticated`, `login()`, `logout()`
AuthGuard	HOC / middleware that redirects unauthenticated users to `/login`
useAuth()	Custom hook returning auth state and functions
SessionManager	Handles token refresh, session expiry detection, logout on 401

5.3 Token Management

Property	Value
Token type	JWT (stored in httpOnly cookie)
Storage	httpOnly, Secure, SameSite=Strict cookie — never in localStorage
Expiry	Access token: 1 hour. Refresh: 7 days
Refresh	Automatic silent refresh when access token has < 5 min remaining
Logout	Delete cookie + `POST /api/v1/auth/logout` (revoke server-side session)

5.4 Auth State Management



5.5 Protected Route Pattern

```
// middleware.ts (Next.js middleware)

import { NextRequest, NextResponse } from "next/server";

const PUBLIC_ROUTES = ["/login"];

export function middleware(request: NextRequest) {
  const token = request.cookies.get("siem_token");
  const isPublicRoute = PUBLIC_ROUTES.includes(request.nextUrl.pathname);

  if (!token && !isPublicRoute) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  if (token && isPublicRoute) {
    return NextResponse.redirect(new URL("/", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};
```

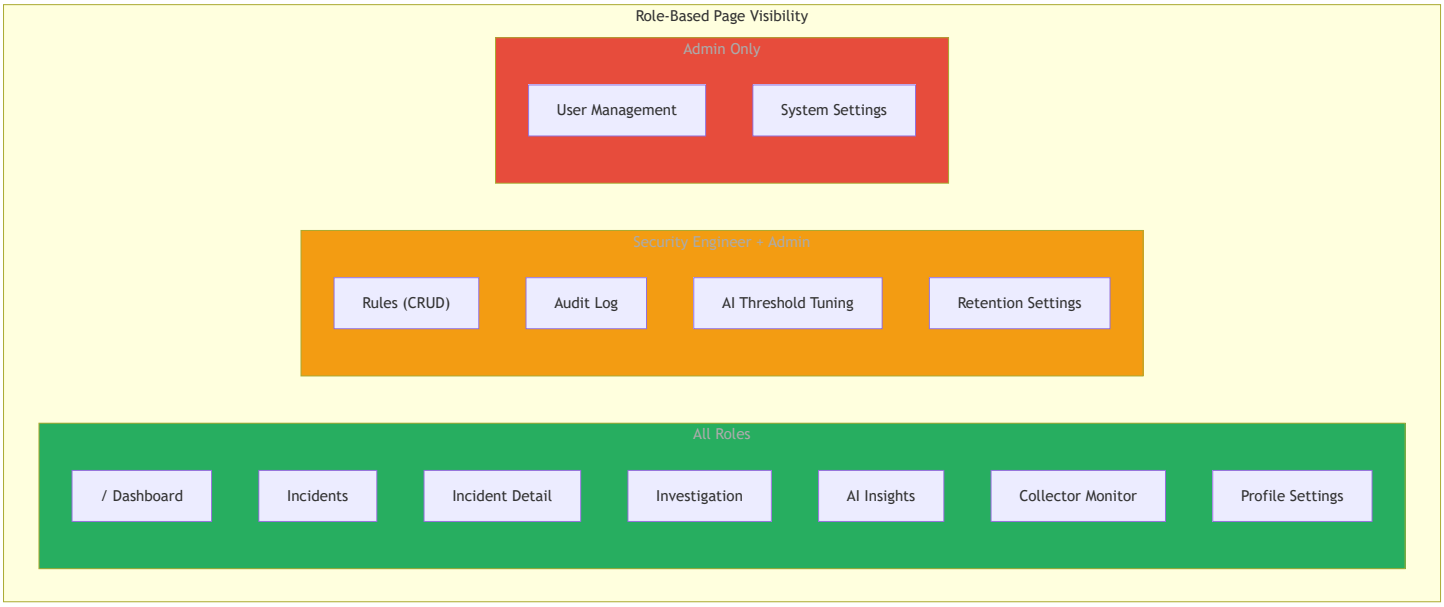
6. Role-Based UI

6.1 Roles

Role	Description
Admin	Full system access. User management, settings, all CRUD
Security Engineer	Rule management, AI threshold tuning, incident management. No user management

Role	Description
SOC Analyst	View dashboards, investigate incidents, search events. No rule editing or user management

6.2 Role-Based Visibility Matrix

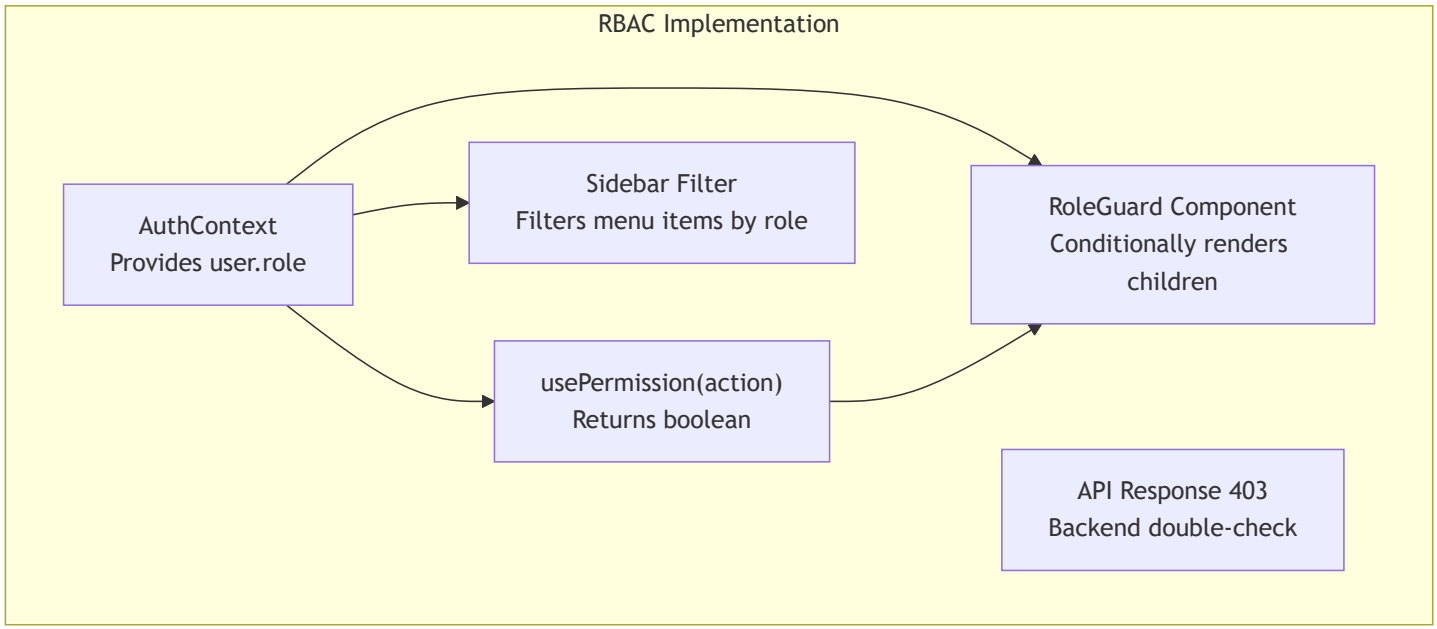


6.3 UI Element Visibility by Role

UI Element	Admin	Security Engineer	SOC Analyst
Sidebar: Dashboard	✅	✅	✅
Sidebar: Incidents	✅	✅	✅
Sidebar: Investigation	✅	✅	✅
Sidebar: Rules	✅	✅	✅ (read-only)
Sidebar: AI Insights	✅	✅	✅
Sidebar: Collector	✅	✅	✅
Sidebar: Settings	✅	✅	✅ (profile only)
Sidebar: Audit Log	✅	✅	❌ Hidden
Sidebar: Users	✅	❌ Hidden	❌ Hidden
Incident: Change status	✅	✅	✅

UI Element	Admin	Security Engineer	SOC Analyst
Incident: Assign analyst	✅	✅	❌ Hidden
Incident: Add note	✅	✅	✅
Rule: Create button	✅	✅	❌ Hidden
Rule: Edit button	✅	✅	❌ Hidden
Rule: Delete button	✅	❌ Hidden	❌ Hidden
Rule: Enable/Disable	✅	✅	❌ Hidden
Rule: Import/Export	✅	✅	Export only
Rule: Test (dry-run)	✅	✅	✅
Settings: AI threshold	✅	✅	❌ Hidden
Settings: Retention	✅	❌ Hidden	❌ Hidden

6.4 RBAC Implementation



```ts
// hooks/usePermission.ts

type Permission =
  | "incidents.assign"
  | "rules.create"
  | "rules.edit"
  | "rules.delete"
  | "rules.enable"
  | "users.manage"
  | "settings.system"
  | "settings.ai_threshold"
  | "audit.view";

const ROLE_PERMISSIONS: Record<UserRole, Permission[]> = {
  admin: [
    "incidents.assign",
    "rules.create", "rules.edit", "rules.delete", "rules.enable",
    "users.manage",
    "settings.system", "settings.ai_threshold",
    "audit.view",
  ],
  security_engineer: [
    "incidents.assign",
    "rules.create", "rules.edit", "rules.enable",
    "settings.ai_threshold",
    "audit.view",
  ],
  soc_analyst: [],
};

function usePermission(permission: Permission): boolean {
  const { user } = useAuth();
  if (!user) return false;
  return ROLE_PERMISSIONS[user.role].includes(permission);
}
```

```tsx
// components/RoleGuard.tsx

function RoleGuard({
  permission,
  children,
  fallback = null
}: {
  permission: Permission;
  children: React.ReactNode;
  fallback?: React.ReactNode;
}) {
  const hasPermission = usePermission(permission);
  return hasPermission ? <>{children}</> : <>{fallback}</>;
}

// Usage:
// <RoleGuard permission="rules.create">
//   <Button>Create Rule</Button>
// </RoleGuard>
```

7. Folder Structure

7.1 Complete Project Structure

```
frontend/
├── public/
│   ├── favicon.ico
│   ├── logo.svg
│   └── manifest.json
│
├── src/
│   ├── app/                                # Next.js App Router
│   │   ├── layout.tsx                      # Root layout (fonts, metadata)
│   │   ├── page.tsx                        # Redirect to /login or /
│   │   ├── globals.css                     # Global styles + CSS variables
│   │   │
│   │   ├── (auth)/                         # Auth group (no layout shell)
│   │   │   └── login/
│   │   │       └── page.tsx                # Login page
│   │   │
│   │   └── (dashboard)/                    # Dashboard group (with AppShell)
│   │       ├── layout.tsx                  # AppShell layout (sidebar+header)
│   │       ├── page.tsx                    # Dashboard (/)
│   │       │
│   │       ├── incidents/
│   │       │   ├── page.tsx                # Incident list
│   │       │   └── [id]/
│   │       │       └── page.tsx            # Incident detail
│   │       │
│   │       ├── investigate/
│   │       │   └── page.tsx                # Log investigation/search
│   │       │
│   │       ├── rules/
│   │       │   ├── page.tsx                # Rules list
│   │       │   ├── new/
│   │       │   │   └── page.tsx            # Create rule
│   │       │   └── [id]/
│   │       │       └── edit/
│   │       │           └── page.tsx        # Edit rule
│   │       │
│   │       ├── ai-insights/
│   │       │   └── page.tsx                # AI insights
```

```
│   │       │
│   │       ├── collector/
│   │       │   └── page.tsx                # Collector monitor
│   │       │
│   │       ├── settings/
│   │       │   ├── page.tsx                # Settings overview
│   │       │   └── users/
│   │       │       └── page.tsx            # User management
│   │       │
│   │       └── audit/
│   │           └── page.tsx                # Audit log
│   │
│   ├── components/                         # Shared UI components
│   │   ├── layout/
│   │   │   ├── AppShell.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   ├── Header.tsx
│   │   │   ├── PageWrapper.tsx
│   │   │   └── Breadcrumbs.tsx
│   │   │
│   │   ├── data-display/
│   │   │   ├── DataTable.tsx
│   │   │   ├── KPICard.tsx
│   │   │   ├── StatusBadge.tsx
│   │   │   ├── SeverityBadge.tsx
│   │   │   ├── RiskScoreBar.tsx
│   │   │   ├── EventTimeline.tsx
│   │   │   ├── JSONViewer.tsx
│   │   │   └── EmptyState.tsx
│   │   │
│   │   ├── charts/
│   │   │   ├── AreaChart.tsx
│   │   │   ├── DonutChart.tsx
│   │   │   ├── BarChart.tsx
│   │   │   ├── GaugeChart.tsx
│   │   │   ├── Heatmap.tsx
│   │   │   ├── WaterfallChart.tsx
│   │   │   ├── Histogram.tsx
│   │   │   └── ChartTheme.ts
│   │   │
│   │   ├── inputs/
│   │   │   ├── SearchBar.tsx
│   │   │   ├── FilterPanel.tsx
```

```
│   │   │   ├── TimeRangePicker.tsx
│   │   │   ├── YAMLEditor.tsx
│   │   │   ├── NoteInput.tsx
│   │   │   └── Select.tsx
│   │   │
│   │   ├── feedback/
│   │   │   ├── Toast.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── LoadingSpinner.tsx
│   │   │   ├── Skeleton.tsx
│   │   │   └── NotificationBadge.tsx
│   │   │
│   │   └── auth/
│   │       ├── AuthProvider.tsx
│   │       ├── AuthGuard.tsx
│   │       ├── RoleGuard.tsx
│   │       └── LoginForm.tsx
│   │
│   ├── hooks/                          # Custom React hooks
│   │   ├── useAuth.ts
│   │   ├── usePermission.ts
│   │   ├── useWebSocket.ts
│   │   ├── usePolling.ts
│   │   ├── usePagination.ts
│   │   ├── useFilters.ts
│   │   ├── useDebounce.ts
│   │   └── useMediaQuery.ts
│   │
│   ├── services/                       # API and WebSocket clients
│   │   ├── api/
│   │   │   ├── client.ts               # Axios instance (base URL, interceptors)
│   │   │   ├── auth.ts                 # POST /auth/login, /auth/logout
│   │   │   ├── incidents.ts            # CRUD /incidents
│   │   │   ├── events.ts               # GET /events
│   │   │   ├── rules.ts                # CRUD /rules
│   │   │   ├── dashboard.ts            # GET /dashboard/*
│   │   │   ├── collector.ts            # GET /collector/status
│   │   │   ├── users.ts                # CRUD /users
│   │   │   └── audit.ts                # GET /audit
│   │   │
│   │   └── websocket/
│   │       ├── WebSocketClient.ts      # WebSocket connection manager
│   │       └── channels.ts             # Channel subscriptions
```

```
│   │
│   ├── stores/                          # Zustand state management
│   │   ├── authStore.ts                 # User, token, role
│   │   ├── incidentStore.ts             # Active incident filters, selected
│   │   ├── dashboardStore.ts            # Dashboard widget data
│   │   ├── notificationStore.ts         # Unread notifications
│   │   └── uiStore.ts                   # Sidebar collapsed, theme, etc.
│   │
│   ├── types/                           # TypeScript type definitions
│   │   ├── incident.ts
│   │   ├── alert.ts
│   │   ├── rule.ts
│   │   ├── event.ts
│   │   ├── user.ts
│   │   ├── collector.ts
│   │   ├── dashboard.ts
│   │   ├── auth.ts
│   │   └── api.ts                       # API request/response types
│   │
│   ├── utils/                           # Utility functions
│   │   ├── formatters.ts                # Date, number, duration formatting
│   │   ├── severity.ts                  # Severity color/label mapping
│   │   ├── permissions.ts               # ROLE_PERMISSIONS map
│   │   ├── constants.ts                 # API URLs, time ranges, etc.
│   │   └── validators.ts                # Form validation helpers
│   │
│   └── styles/                          # Shared style utilities
│       ├── variables.css                # CSS custom properties (design tokens)
│       ├── typography.css               # Font definitions
│       ├── animations.css               # Shared animations (pulse, fade, slide)
│       └── components.css               # Shared component styles
│
├── middleware.ts                         # Next.js middleware (auth redirect)
├── next.config.js                       # Next.js configuration
├── tsconfig.json                        # TypeScript configuration
├── package.json
├── .env.local                           # Environment variables
├── .env.example
├── Dockerfile
└── README.md
```

7.2 Design Tokens (CSS Variables)

```css
/* src/styles/variables.css */

:root {
  /* --- Color Palette --- */
  --color-bg-primary: #0f0f1a;
  --color-bg-secondary: #1a1a2e;
  --color-bg-tertiary: #16213e;
  --color-bg-card: #1e1e36;
  --color-bg-hover: #252542;

  --color-text-primary: #ecf0f1;
  --color-text-secondary: #95a5a6;
  --color-text-muted: #7f8c8d;

  --color-accent-primary: #3498db;
  --color-accent-secondary: #2ecc71;
  --color-accent-warning: #f39c12;
  --color-accent-danger: #e74c3c;

  /* --- Severity Colors --- */
  --color-severity-critical: #e74c3c;
  --color-severity-high: #e67e22;
  --color-severity-medium: #f1c40f;
  --color-severity-low: #3498db;
  --color-severity-info: #95a5a6;

  /* --- Status Colors --- */
  --color-status-online: #2ecc71;
  --color-status-degraded: #f39c12;
  --color-status-offline: #e74c3c;

  /* --- Spacing --- */
  --space-xs: 4px;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --space-xl: 32px;
  --space-2xl: 48px;

  /* --- Border Radius --- */
  --radius-sm: 4px;
```

```
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-full: 9999px;

  /* --- Shadows --- */
  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.3);
  --shadow-md: 0 4px 6px rgba(0, 0, 0, 0.4);
  --shadow-lg: 0 10px 15px rgba(0, 0, 0, 0.5);
  --shadow-glow-critical: 0 0 12px rgba(231, 76, 60, 0.4);
  --shadow-glow-accent: 0 0 12px rgba(52, 152, 219, 0.3);

  /* --- Typography --- */
  --font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace;
  --font-size-xs: 0.75rem;
  --font-size-sm: 0.875rem;
  --font-size-md: 1rem;
  --font-size-lg: 1.125rem;
  --font-size-xl: 1.5rem;
  --font-size-2xl: 2rem;
  --font-size-3xl: 2.5rem;

  /* --- Sidebar --- */
  --sidebar-width: 250px;
  --sidebar-collapsed-width: 64px;

  /* --- Transitions --- */
  --transition-fast: 150ms ease;
  --transition-normal: 250ms ease;
  --transition-slow: 400ms ease;

  /* --- Z-Index Scale --- */
  --z-sidebar: 100;
  --z-header: 200;
  --z-modal: 300;
  --z-toast: 400;
}
```

7.3 Key Dependencies

Dependency	Version	Purpose
next	14+	React SSR framework with App Router
react	18+	UI library
typescript	5.x	Type safety
recharts	2.x	Chart components (Area, Bar, Donut, Line)
zustand	4.x	Lightweight state management
axios	1.x	HTTP client for REST API
@monaco-editor/react	4.x	YAML rule editor
react-hot-toast	2.x	Toast notifications
date-fns	3.x	Date formatting and manipulation
clsx	2.x	Conditional className composition
lucide-react	latest	Icon library

7.4 Environment Variables

```
# .env.local

# Backend API
NEXT_PUBLIC_API_URL=http://localhost:3000
NEXT_PUBLIC_WS_URL=ws://localhost:3000/ws

# App Configuration
NEXT_PUBLIC_APP_NAME=AI-SIEM
NEXT_PUBLIC_REFRESH_INTERVAL=30000
NEXT_PUBLIC_WS_RECONNECT_INTERVAL=5000
```

7.5 Build and Run

Command	Description
`npm run dev`	Development server on `http://localhost:3001`

Command	Description
`npm run build`	Production build
`npm run start`	Start production server
`npm run lint`	ESLint check
`npm run type-check`	TypeScript compilation check

> **This document is governed by [ADR-001] (file:///d:/AI%20SIEM/docs/architecture/decisions/ADR-001-modular-monolith.md). The Frontend is Process 4 in the platform architecture. It is a Next.js 14 application using the App Router, communicating with the backend via REST API and WebSocket. Authentication uses JWT in httpOnly cookies. Role-based visibility is enforced at the UI level and double-checked by the backend API.**
>
> **Document Version**: 1.0
> **Last Updated**: 2026-07-18